CSAI 204 – Operating Systems

Lab 5 Report

Priority Scheduling Mechanism in xv6

1. Objective

The purpose of this lab was to enhance the xv6 scheduler by introducing process priorities and weighted CPU allocation. Two new system calls were added to allow user programs to modify and retrieve process priorities dynamically. The scheduler was redesigned so higher-priority processes receive more execution opportunities while maintaining fairness between all runnable processes.

2. Implementation Steps

Step 1 – Extending the Process Structure
The process control block was updated with additional fields responsible for storing process priority information and tracking how often each process executes during a scheduling cycle.

Step 2 – Default Initialization
During process allocation, the scheduler-related fields were initialized with default values to ensure newly created processes start in a consistent state before any priority modifications occur.

Step 3 – Defining New System Call IDs
Unique identifiers were assigned to the new scheduler-related system calls so they could be linked through the kernel syscall dispatcher.

Step 4 – Updating the Dispatch Table
The syscall registration table was extended with the new handlers, allowing user-space requests to reach the correct kernel-level implementation.

Step 5 – Kernel Prototype Declarations
Function declarations for manipulating process priorities were inserted into the kernel header files to support communication between scheduler components.

Step 6 – Implementing Priority Functions
Kernel functions were developed to safely update and retrieve process priorities. Synchronization mechanisms were used to avoid inconsistent scheduler behavior during concurrent execution.

Step 7 – Creating Syscall Wrappers
Wrapper functions inside sysproc.c handled argument retrieval from user space and passed the values to the scheduler management functions.

Step 8 – Exposing Calls to User Programs
User-level declarations and syscall stubs were added so applications running in xv6 could directly invoke the new scheduling system calls.

Step 9 – Modifying the Scheduler
The scheduler logic was redesigned to implement weighted round-robin behavior. Processes with higher priority values were selected more frequently, while periodic counter resets prevented starvation of lower-priority tasks.

3. Test Program

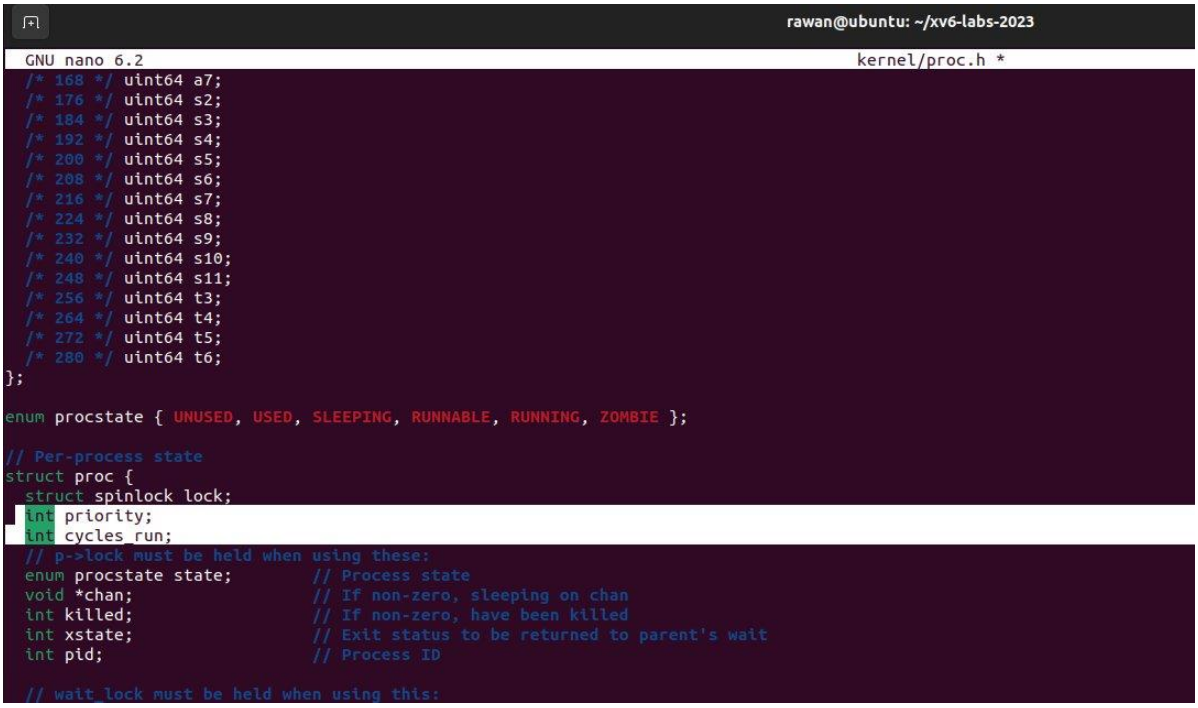
A validation program was developed to evaluate scheduler behavior under different priorities. The application created multiple child processes with distinct priority levels and monitored their execution order. Results demonstrated that higher-priority processes consumed more CPU time compared to lower-priority ones.

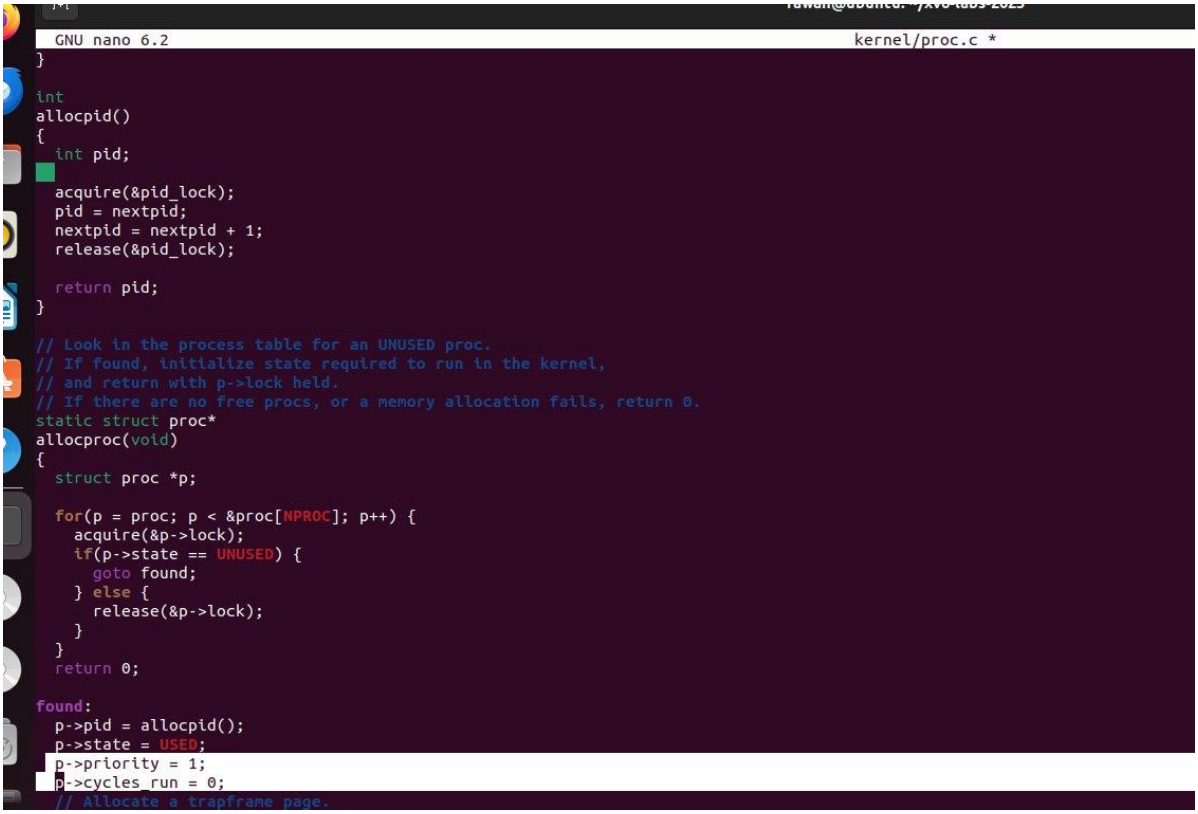
4. Test Results

The execution logs confirmed that the modified scheduler correctly distributed processor time according to process priority. High-priority processes executed more frequently during each cycle, while lower-priority processes still received execution opportunities after counter resets.

5. Screenshots

The following screenshots illustrate the source code modifications, scheduler implementation, Makefile configuration, and execution results produced during testing.





```
  GNU nano 6.2                                                kernel/proc.c *
}

int
allocpid()
{
  int pid;

  acquire(&pid_lock);
  pid = nextpid;
  nextpid = nextpid + 1;
  release(&pid_lock);

  return pid;
}

// Look in the process table for an UNUSED proc.
// If found, initialize state required to run in the kernel,
// and return with p->lock held.
// If there are no free procs, or a memory allocation fails, return 0.
static struct proc*
allocproc(void)
{
  struct proc *p;

  for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if(p->state == UNUSED) {
      goto found;
    } else {
      release(&p->lock);
    }
  }
  return 0;

found:
  p->pid = allocpid();
  p->state = USED;
  p->priority = 1;
  p->cycles_run = 0;
  // Allocate a trapframe page.
```

```
rawan@ubuntu: ~/xv6-labs-2023
  GNU nano 6.2                                                kernel/syscall.h *
// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup    10
#define SYS_getpid 11
#define SYS_sbrk   12
#define SYS_sleep  13
#define SYS_uptime 14
#define SYS_open   15
#define SYS_write  16
#define SYS_mknod  17
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
#define SYS_setprio 23
#define SYS_getprio 24
```

rawan@ubuntu: ~/xv6-labs-2023

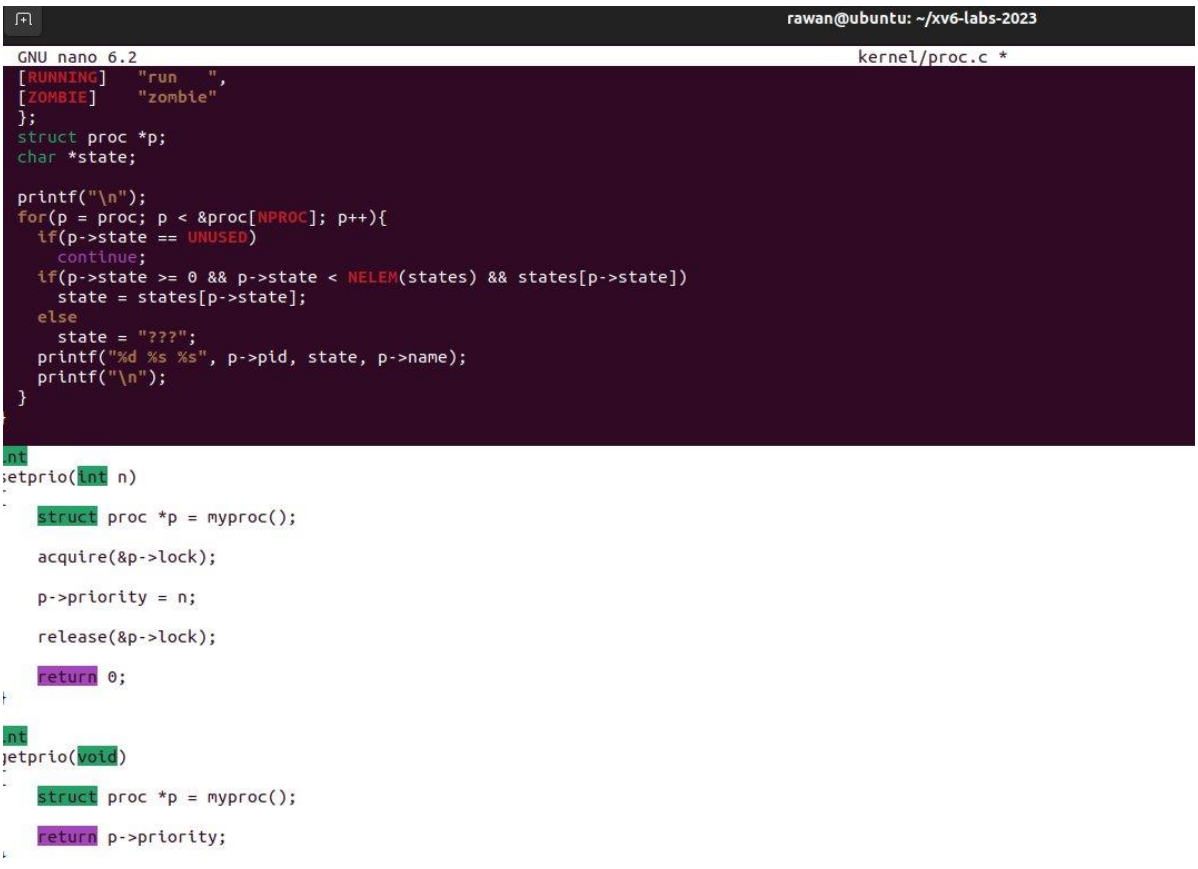
GNU nano 6.2 kernel/syscall.c *

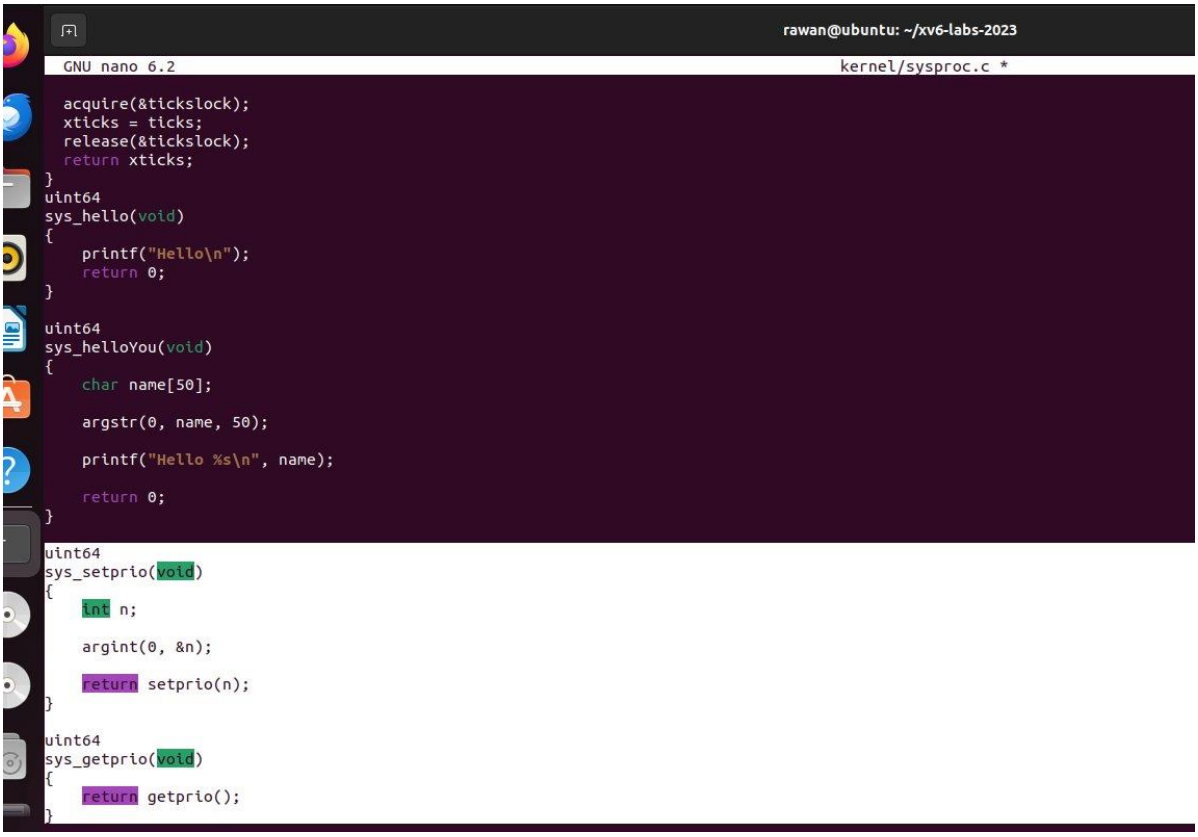
```
// Copies into buf, at most max.
// Returns string length if OK (including nul), -1 if error.
int
argstr(int n, char *buf, int max)
{
  uint64 addr;
  argaddr(n, &addr);
  return fetchstr(addr, buf, max);
}

// Prototypes for the functions that handle system calls.
extern uint64 sys_fork(void);
extern uint64 sys_hello(void);
extern uint64 sys_helloYou(void);
extern uint64 sys_exit(void);
extern uint64 sys_wait(void);
extern uint64 sys_pipe(void);
extern uint64 sys_read(void);
extern uint64 sys_kill(void);
extern uint64 sys_exec(void);
extern uint64 sys_fstat(void);
extern uint64 sys_chdir(void);
extern uint64 sys_dup(void);
extern uint64 sys_getpid(void);
extern uint64 sys_sbrk(void);
extern uint64 sys_sleep(void);
extern uint64 sys_uptime(void);
extern uint64 sys_open(void);
extern uint64 sys_write(void);
extern uint64 sys_mknod(void);
extern uint64 sys_unlink(void);
extern uint64 sys_link(void);
extern uint64 sys_mkdir(void);
extern uint64 sys_close(void);
extern uint64 sys_setprio(void);
extern uint64 sys_getprio(void);

// An array mapping syscall numbers from syscall.h
```

GNU nano 6.2 kernel/syscall.c *

```
extern uint64 sys_mkdir(void);
extern uint64 sys_close(void);
extern uint64 sys_setprio(void);
extern uint64 sys_getprio(void);

// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]   sys_fstat,
[SYS_chdir]   sys_chdir,
[SYS_dup]     sys_dup,
[SYS_getpid]  sys_getpid,
[SYS_sbrk]    sys_sbrk,
[SYS_sleep]   sys_sleep,
[SYS_uptime]  sys_uptime,
[SYS_open]    sys_open,
[SYS_write]   sys_write,
[SYS_mknod]   sys_mknod,
[SYS_unlink]  sys_unlink,
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
[SYS_hello] sys_hello,
[SYS_helloYou] sys_helloYou,
[SYS_setprio] sys_setprio,
[SYS_getprio] sys_getprio,
};
```


rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2
kernel/defs.h *
void sleep(void*, struct spinlock*);
void userinit(void);
int wait(uint64);
void wakeup(void*);
void yield(void);
int either_copyout(int user_dst, uint64 dst, void *src, uint64 len);
int either_copyin(void *dst, int user_src, uint64 src, uint64 len);
void procdump(void);
int setprio(int);
int getprio(void);
// swtch.S
void swtch(struct context*, struct context*);
// spinlock.c
void acquire(struct spinlock*);


rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2
kernel/proc.c *
[RUNNING] "run ",
[ZOMBIE] "zombie"
};
struct proc *p;
char *state;
printf("\n");
for(p = proc; p < &proc[NPROC]; p++){
if(p->state == UNUSED)
continue;
if(p->state >= 0 && p->state < NELEM(states) && states[p->state])
state = states[p->state];
else
state = "???";
printf("%d %s %s", p->pid, state, p->name);
printf("\n");
}
int
setprio(int n)
{
struct proc *p = myproc();
acquire(&p->lock);
p->priority = n;
release(&p->lock);
return 0;
}
int
getprio(void)
{
struct proc *p = myproc();
return p->priority;
}


rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2
kernel/sysproc.c *
acquire(&tickslock);
xticks = ticks;
release(&tickslock);
return xticks;
}
uint64
sys_hello(void)
{
printf("Hello\n");
return 0;
}
uint64
sys_helloYou(void)
{
char name[50];
argstr(0, name, 50);
printf("Hello %s\n", name);
return 0;
}
uint64
sys_setprio(void)
{
int n;
argint(0, &n);
return setprio(n);
}
uint64
sys_getprio(void)
{
return getprio();
}


rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2
user/user.h *
struct stat;
// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int*);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(const char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
int hello(void);
int helloYou(char *);
int setprio(int);
int getprio(void);

rawan@ubuntu: ~/xv6-labs-2023

GNU nano 6.2 user/usys.pl *

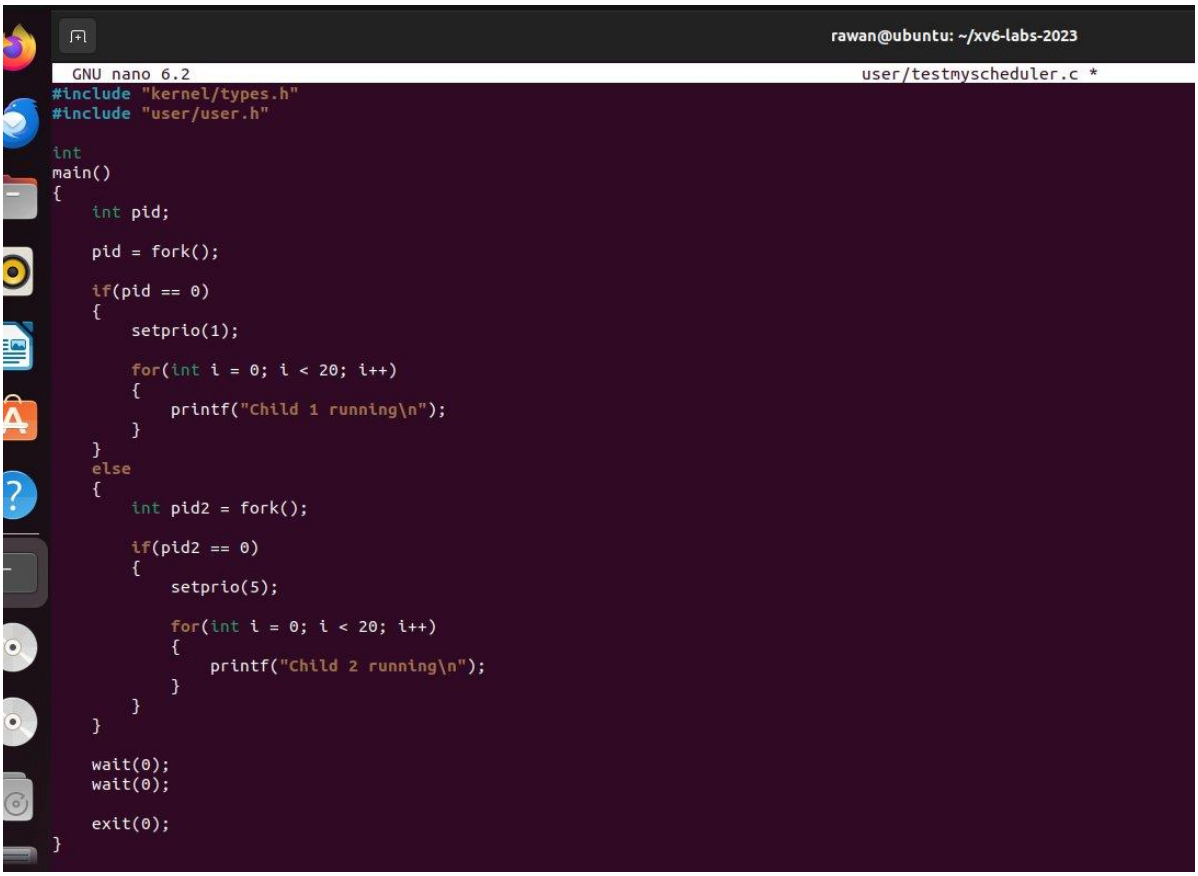
```
# Generate usys.S, the stubs for syscalls.

print "# generated by usys.pl - do not edit\n";

print "#include \"kernel/syscall.h\"\n";

sub entry {
    my $name = shift;
    print ".global $name\n";
    print "${name}:\n";
    print " li a7, SYS_${name}\n";
    print " ecall\n";
    print " ret\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");
entry("kill");
entry("exec");
entry("open");
entry("mknod");
entry("unlink");
entry("fstat");
entry("link");
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("sbrk");
entry("sleep");
entry("uptime");
entry("hello");
entry("helloYou");
entry("setprio");
entry("getprio");
```

rawan@ubuntu: ~/xv6-labs-2023

GNU nano 6.2 kernel/proc.c *

```
      return -1;
    }

    // Wait for a child to exit.
    sleep(p, &wait_lock);  //DOC: wait-sleep
  }
}

// Per-CPU process scheduler.
// Each CPU calls scheduler() after setting itself up.
// Scheduler never returns.  It loops, doing:
//  - choose a process to run.
//  - swtch to start running that process.
//  - eventually that process transfers control
//    via swtch back to the scheduler.
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    // The most recent process to run may have had interrupts
    // turned off; enable them to avoid a deadlock if all
    // processes are waiting.
    intr_on();

    for(p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
      if(p->state == RUNNABLE && p->cycles_run < p->priority) {
        // Switch to chosen process.  It is the process's job
        // to release its lock and then reacquire it
        // before jumping back to us.
        p->state = RUNNING;
        c->proc = p;
```

```
    intr_on();

    for(p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
      if(p->state == RUNNABLE && p->cycles_run < p->p
        // Switch to chosen process.  It is the proce
        // to release its lock and then reacquire it
        // before jumping back to us.
        p->state = RUNNING;
        c->proc = p;
        p->cycles_run++;
        swtch(&c->context, &p->context);
```




rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2
Makefile *
RECIOUS: %.o
ROGS=\
$U/_cat\
$U/_echo\
$U/_forktest\
$U/_grep\
$U/_init\
$U/_kill\
$U/_ln\
$U/_ls\
$U/_mkdir\
$U/_rm\
$U/_sh\
$U/_stressfs\
$U/_usertests\
$U/_grind\
$U/_wc\
$U/_zombie\
$U/_pingpong\
$U/_testmyscheduler\
eq ($(LAB),$(filter $(LAB), lock))
ROGS += \
$U/_stats
dif


ph barrier
# try to generate a unique GDB por
GDBPORT = $(shell expr `id -u` % 5
# QEMU's gdb stub command line cha
QEMUGDB = $(shell if $(QEMU) -help
then echo "-gdb tcp::$(GDB
else echo "-s -p $(GDBPORT
ifndef CPUS
CPUS := 1
endif
ifeq ($(LAB),fs)
CPUS := 1

6. Conclusion

This lab demonstrated how scheduling policies in xv6 can be extended through kernel modifications and custom system calls. The weighted scheduling approach improved CPU allocation flexibility and highlighted the relationship between process management, synchronization, and operating system design.